Pagewise: Learn the Claude API by Building One Project

A reference guide in Python with uv. Written October 2026. Keep it open next to your editor.

How to use this guide

You build one project, **Pagewise**, a study assistant. Every chapter adds one feature to the same folder. Each chapter has the same shape: **Problem** (why we need it), **Concept** (plain words), **Code**, **Run it**, and **Watch out** (common mistakes).

Status tags. *Stable* means a core API that rarely changes. *Beta* means a feature whose syntax, tool version strings or beta headers have moved recently. For Beta chapters, copy the current strings from the docs page named in the chapter. I wrote this code from my knowledge of the SDK and the docs; run every chapter, and treat an error as a signal to check the docs, not as your mistake.

Budget rules for a $5 plan (prices below are from a third-party listing: Haiku 4.5 is about $1 per million input tokens and $5 per million output tokens; confirm on Anthropic's pricing page):

- Use Haiku 4.5 (the default in this guide). Switch to a bigger model only for a comparison call.
- Always set `max_tokens`, and cap every agent loop with `max_turns`.
- Test on small text, not whole books.
- After each Act, open Console, Usage, and look at what you spent.

The map

Act	Chapters	Theme
1. Talking	1 to 6	Messages, prompting, long context, thinking, structured output, citations
2. Efficiency	7 to 8	Token counting, prompt caching, batch
3. Hands	9 to 12	Tool use, web search, code execution, Files API

Act	Chapters	Theme
4. Library (RAG)	13 to 15	Retrieval, contextual retrieval, agentic search
5. Context engineering	16 to 19	Attention budget, compaction, memory, tool design
6. Agents	20 to 27	Agent loop, workflows, Skills and MCP, Agent SDK, harness, loop engineering, sub-agents, evals

---

Chapter 0: Set up the project

Problem. We need a clean project and a safe way to hold the API key.

```
uv init pagewise
cd pagewise
uv add anthropic
```

Create the key in the Claude Console (Settings, API Keys). It starts with `sk-ant-` and is shown once. Put it in your shell, never in code:

```
# macOS / Linux
export ANTHROPIC_API_KEY='sk-ant-...'
# Windows PowerShell
$env:ANTHROPIC_API_KEY='sk-ant-...'
```

Create `client.py`. Every chapter imports from it:

```
from anthropic import Anthropic

client = Anthropic()  # reads ANTHROPIC_API_KEY from the environment
MODEL = 'claude-haiku-4-5-20251001'  # cheap, good for learning


def text_of(msg):
    # a reply is a LIST of blocks; this joins just the text ones
    return ''.join(b.text for b in msg.content if b.type == 'text')
```

Watch out. Never commit the key to git. Add a `.gitignore` with `.env` if you use a `.env` file. If you see an authentication error, the variable is not set in the terminal you are running from.

Act 1: Pagewise learns to talk

Chapter 1: First call, memory, streaming

Status: Stable.

Problem. Say hello to Claude from code, and understand what is really going on.

Concept. You send a list of `messages` (alternating `user` and `assistant`), plus optional `system` instructions. Claude replies with a list of content blocks. A **token** is roughly three quarters of a word, and you pay per token in and out. The API is **stateless**: Claude remembers nothing between calls, so you resend the history yourself.

```
# ch01_first_call.py
from client import client, MODEL, text_of

msg = client.messages.create(
    model=MODEL,
    max_tokens=300,                      # hard cap on the reply length
    system='You are Pagewise, a friendly study assistant.',
    messages=[{'role': 'user', 'content': 'In 2 sentences, what is a context
window?'}],
)
print(text_of(msg))
print(msg.stop_reason)   # end_turn = finished, max_tokens = you cut it off
print(msg.usage)         # input_tokens and output_tokens = your cost

# memory is YOUR job: append both sides and resend
history = [{'role': 'user', 'content': 'My name is Asha and I study
biology.'}]
r1 = client.messages.create(model=MODEL, max_tokens=200, messages=history)
history.append({'role': 'assistant', 'content': r1.content})
history.append({'role': 'user', 'content': 'What do I study?'})
r2 = client.messages.create(model=MODEL, max_tokens=200, messages=history)
print(text_of(r2))

# streaming: print words as they arrive
with client.messages.stream(model=MODEL, max_tokens=200,
```

```
        messages=[{'role': 'user', 'content': 'Explain tokens in 2
sentences.'}]) as s:
    for t in s.text_stream:
        print(t, end='', flush=True)
```

Run it: `uv run python ch01_first_call.py`

Watch out. `stop_reason == 'max_tokens'` means the answer was cut mid-sentence. Messages must alternate roles. The SDK retries some transient errors automatically.

Chapter 2: Prompting that works

Status: Stable.

Problem. Answers are vague or too long.

Concept. Prompt engineering is clear instructions plus structure. Four habits cover most of it: (1) put the role and rules in `system`; (2) wrap material in XML-style tags like `<material>` so Claude can tell your content from your question; (3) show one or two examples of the output you want; (4) say what to do when unsure, so it does not guess.

```
# ch02_prompting.py
from client import client, MODEL, text_of

notes = open('book.txt', encoding='utf-8').read()[:6000]   # keep it small

SYSTEM = '''You are Pagewise, a study assistant for a first-year biology
student.
Rules:
- Explain in plain language, then give one concrete example.
- If the answer is not in the material, say so instead of guessing.
- Keep answers under 120 words.'''

vague = 'Explain this.'
sharp = f'''<material>
{notes}
</material>

<task>Explain the main idea of the material.</task>

<example>
Q: What is osmosis?
```

```
A: Water moving across a membrane toward the saltier side. Example: a raisin
swelling in water.
</example>'''

for p in (vague, sharp):
    r = client.messages.create(model=MODEL, max_tokens=400, system=SYSTEM,
                               messages=[{'role': 'user', 'content': p}])
    print(text_of(r))
    print('-' * 40)
```

Run it and compare the two answers. The first one has nothing to work with.

Watch out. Long, contradictory instructions make results worse. Say what to do, not only what to avoid. Put long documents before the question.

Chapter 3: Long context and PDFs

Status: Stable.

Problem. The student has a whole textbook chapter as a PDF.

Concept. The **context window** is the maximum amount Claude can read in one call (input plus output). Some current models go up to 1M tokens. PDFs are sent as `document` blocks; Claude reads both the text and the pictures (charts, diagrams), which costs more tokens than plain text.

```
# ch03_pdf.py
import base64
from client import client, MODEL, text_of

pdf = base64.standard_b64encode(open('chapter.pdf', 'rb').read()).decode()

r = client.messages.create(
    model=MODEL,
    max_tokens=600,
    messages=[{
        'role': 'user',
        'content': [
            {'type': 'document',
             'source': {'type': 'base64', 'media_type': 'application/pdf',
'data': pdf}},
            {'type': 'text', 'text': 'List the 5 key ideas, one line each.
Mention any chart you see.'},
```

```
        ],
    }],
)
print(text_of(r))
print(r.usage)   # note how large input_tokens is
```

Watch out. File size and page limits apply (check the PDF support page). Bigger context means a bigger bill on every call. Chapter 7 fixes that.

Chapter 4: Thinking and effort

Status: Beta-ish. Syntax differs by model generation; verify on the extended thinking page.

Problem. Pagewise rushes multi-step problems.

Concept. With **extended thinking**, Claude reasons in a separate `thinking` block before answering. You give a token budget for that reasoning. Thinking tokens are billed as output. Newer models add **adaptive thinking** (Claude decides when and how much to think) and an **effort** setting (low to high) to trade quality against cost; their parameter names have changed between releases, so check the docs for the model you use.

```
# ch04_thinking.py
from client import client, MODEL

q = 'A tank fills in 6 hours with pipe A and 9 hours with pipe B. A drain
empties it in 12 hours. With all three open, how long to fill?'

r = client.messages.create(
    model=MODEL,
    max_tokens=4000,                                    # must be bigger than
the budget
    thinking={'type': 'enabled', 'budget_tokens': 2000},  # minimum is 1024
    messages=[{'role': 'user', 'content': q}],
)
for b in r.content:
    if b.type == 'thinking':
        print('THINKING:', b.thinking[:300], '...')
    elif b.type == 'text':
        print('ANSWER:', b.text)
print(r.usage)
```

Watch out. Do not turn thinking on for easy tasks; you pay for it. When you send history back in later turns, keep thinking blocks exactly as returned if tool use is involved.

Chapter 5: Structured output (flashcards your code can use)

Status: Stable (method shown). A native structured-outputs feature also exists; see Watch out.

Problem. We want flashcards as data, not a paragraph.

Concept. Define a **tool** whose input schema is the shape you want, then force Claude to call it with `tool_choice`. The tool call's `input` is your structured data. This works on every model and teaches the tool schema you need again in Chapter 9.

```
# ch05_flashcards.py
import json
from client import client, MODEL

tools = [{
    'name': 'save_flashcards',
    'description': 'Save study flashcards made from the material.',
    'input_schema': {
        'type': 'object',
        'properties': {
            'cards': {
                'type': 'array',
                'items': {
                    'type': 'object',
                    'properties': {'question': {'type': 'string'}, 'answer':
{'type': 'string'}},
                    'required': ['question', 'answer'],
                },
            }
        },
        'required': ['cards'],
    },
}]

notes = open('book.txt', encoding='utf-8').read()[:6000]
r = client.messages.create(
```

```
    model=MODEL, max_tokens=1500, tools=tools,
    tool_choice={'type': 'tool', 'name': 'save_flashcards'},   # force it
    messages=[{'role': 'user', 'content': f'Make 5 flashcards from
this:\n{notes}'}],
)
cards = next(b.input for b in r.content if b.type == 'tool_use')['cards']
json.dump(cards, open('flashcards.json', 'w'), indent=2)
print(len(cards), 'cards saved')
```

Watch out. Anthropic also offers a native structured-outputs feature that guarantees schema-valid JSON, but support depends on the model and endpoint, and the parameter has changed; check the structured outputs docs before relying on it. Always validate the data before using it.

Chapter 6: Citations

Status: Stable.

Problem. Can we trust an answer?

Concept. Give the material as a `document` block with citations enabled. Claude's text blocks then carry `citations` that point to the exact passage used.

```
# ch06_citations.py
from client import client, MODEL

text = open('book.txt', encoding='utf-8').read()[:6000]
doc = {
    'type': 'document',
    'source': {'type': 'text', 'media_type': 'text/plain', 'data': text},
    'title': 'Biology notes',
    'citations': {'enabled': True},
}
r = client.messages.create(
    model=MODEL, max_tokens=500,
    messages=[{'role': 'user', 'content': [doc, {'type': 'text', 'text':
'What is the main claim? Answer in 2 sentences.'}]}],
)
for b in r.content:
    if b.type == 'text':
        print(b.text)
        for c in (b.citations or []):
            print('   SOURCE:', c.cited_text[:100])
```

Watch out. Citations only cover material you supply as documents or search results. Quoted text (`cited_text`) does not count against your output tokens.

Act 2: Pagewise gets cheaper

Chapter 7: Token counting and prompt caching

Status: Stable.

Problem. Every question re-sends the whole book, and you pay again.

Concept. `count_tokens` tells you the size of a prompt before you pay for it. **Prompt caching** stores a big, unchanging prefix: the first call writes it (a bit more than normal price), later calls within the cache lifetime (5 minutes by default, 1 hour optional) read it at a small fraction of the price. Mark the end of the reusable part with `cache_control` .

```
# ch07_cache.py
from client import client, MODEL, text_of

book = open('book.txt', encoding='utf-8').read()

n = client.messages.count_tokens(model=MODEL, messages=[{'role': 'user',
'content': book}])
print('book is', n.input_tokens, 'tokens')

system = [
    {'type': 'text', 'text': 'You are Pagewise. Answer only from the book.'},
    {'type': 'text', 'text': book, 'cache_control': {'type': 'ephemeral'}},
# cache up to here
    # for a 1-hour cache: {'type': 'ephemeral', 'ttl': '1h'}
]
for q in ['What is chapter 1 about?', 'Name one key term.', 'Give a quiz
question.']:
    r = client.messages.create(model=MODEL, max_tokens=200, system=system,
                               messages=[{'role': 'user', 'content': q}])
    u = r.usage
    print(q, '| written:', u.cache_creation_input_tokens, '| read:',
u.cache_read_input_tokens)
```

Run it. The first call should show tokens *written*; the next two should show tokens *read*.

Watch out. There is a minimum prompt length before caching kicks in (it varies by model, roughly 1,024 to 4,096 tokens). If both numbers are zero, your text is too short. Change anything before the cache marker and the cache is lost, so keep the stable part first.

Chapter 8: Batch processing

Status: Stable.

Problem. Make quiz questions for 200 chapters overnight, cheaply.

Concept. The Message Batches API runs many requests asynchronously at about half price. You submit a list with a `custom_id` each, poll until it ends, then read results (order is not guaranteed, so match by `custom_id`).

```
# ch08_batch.py
import time
from client import client, MODEL

chapters = {'ch1': 'Cells are the basic unit of life.', 'ch2': 'DNA stores
genetic information.'}

batch = client.messages.batches.create(requests=[
    {'custom_id': cid,
     'params': {'model': MODEL, 'max_tokens': 200,
                'messages': [{'role': 'user', 'content': f'Write 1 quiz
question about: {txt}'}]}}
    for cid, txt in chapters.items()
])
print('batch id:', batch.id)

while client.messages.batches.retrieve(batch.id).processing_status !=
'ended':
    time.sleep(30)

for item in client.messages.batches.results(batch.id):
    if item.result.type == 'succeeded':
        print(item.custom_id, item.result.message.content[0].text)
    else:
        print(item.custom_id, 'failed:', item.result.type)
```

Watch out. Batches are for work that can wait (usually done within an hour, sometimes longer). Do not use them for chat. Test with 2 items before sending 2,000.

Act 3: Pagewise gets hands

Chapter 9: Tool use (and your reusable loop)

Status: Stable.

Problem. Pagewise can talk but cannot do anything: look up your course glossary, read a file, check a score.

Concept. You describe **tools** (name, description, input schema). Claude never runs them. When it wants one, it stops with `stop_reason == 'tool_use'` and gives you a `tool_use` block containing the tool name and arguments. *You* run the function, then send back a

`tool_result` block with the matching `tool_use_id`. Claude then continues. Repeat until it stops asking. That cycle is the heart of every agent, so we put it in a reusable file now.

```
# toolkit.py  (used by many later chapters)
from client import client, MODEL

USAGE = {'input': 0, 'output': 0, 'calls': 0}


def cost_usd(price_in=1.0, price_out=5.0):
    # prices are per million tokens; update them from the pricing page
    return USAGE['input'] / 1e6 * price_in + USAGE['output'] / 1e6 *
price_out


def run_with_tools(messages, tools, handlers, system=None, max_turns=8,
max_tokens=1024):
    for turn in range(max_turns):
        kwargs = dict(model=MODEL, max_tokens=max_tokens, messages=messages,
tools=tools)
        if system:
            kwargs['system'] = system
        resp = client.messages.create(**kwargs)
        USAGE['input'] += resp.usage.input_tokens
        USAGE['output'] += resp.usage.output_tokens
        USAGE['calls'] += 1
        messages.append({'role': 'assistant', 'content': resp.content})
        if resp.stop_reason != 'tool_use':
            return resp
        results = []
        for b in resp.content:
            if b.type != 'tool_use':
                continue
            try:
                out, err = handlers[b.name](**b.input), False
            except Exception as e:
                out, err = f'Error: {e}', True
            item = {'type': 'tool_result', 'tool_use_id': b.id, 'content':
str(out)}
            if err:
```

```
                item['is_error'] = True
            results.append(item)
        messages.append({'role': 'user', 'content': results})
    raise RuntimeError(f'stopped after {max_turns} turns')
```

```
# ch09_tools.py
from client import text_of
from toolkit import run_with_tools

GLOSSARY = {'mitosis': 'Cell division that makes two identical cells.',
            'osmosis': 'Water moving across a membrane toward higher solute
concentration.'}


def lookup_term(term):
    return GLOSSARY.get(term.lower(), 'not found in glossary')


tools = [{
    'name': 'lookup_term',
    'description': 'Look up a biology term in the course glossary. Use it
before explaining any term.',
    'input_schema': {'type': 'object',
                     'properties': {'term': {'type': 'string', 'description':
'one term, for example mitosis'}},
                     'required': ['term']},
}]

msgs = [{'role': 'user', 'content': 'What does mitosis mean in our course?'}]
final = run_with_tools(msgs, tools, {'lookup_term': lookup_term})
print(text_of(final))
print(len(msgs), 'messages in the conversation')   # user,
assistant(tool_use), user(tool_result), assistant
```

Watch out. The description is the most important field: it is how Claude decides when to use the tool. Always return tool results, even errors (with `is_error`), or the conversation breaks. The `max_turns` cap protects your wallet from runaway loops. Tool results are untrusted input; never let text inside a result act as instructions to your code.

Chapter 10: Web search and web fetch

Status: web search Stable; web fetch Beta (verify the tool version and beta header on the web fetch tool page).

Problem. Pagewise only knows what it was trained on.

Concept. These are **server tools**: Anthropic runs them for you, so there is no handler to write. Claude decides to search, reads results, and answers with citations. You can limit it with `max_uses`. Web search has a per-search fee on top of token costs (check pricing), and an organization admin may need to enable it in Console settings.

```
# ch10_web.py
from client import client, MODEL

r = client.messages.create(
    model=MODEL, max_tokens=1000,
    tools=[{'type': 'web_search_20250305', 'name': 'web_search', 'max_uses':
2}],
    messages=[{'role': 'user', 'content': 'What is new in the field of CRISPR
gene editing this year? 3 bullets.'}],
)
for b in r.content:
    if b.type == 'text':
        print(b.text)
print(r.usage)   # includes how many searches were run

# web fetch (Beta): read one page you already have a URL for
r2 = client.beta.messages.create(
    model=MODEL, max_tokens=1000,
    betas=['web-fetch-2025-09-10'],
    tools=[{'type': 'web_fetch_20250910', 'name': 'web_fetch', 'max_uses':
1}],
    messages=[{'role': 'user', 'content': 'Summarize https://example.com in
one sentence.'}],
)
print([b.type for b in r2.content])
```

Watch out. Search results count as input tokens in later turns. Keep `max_uses` small. Treat web content as untrusted (prompt injection).

Chapter 11: Code execution

Status: Beta. Tool version strings change; copy the current one from the code execution tool page.

Problem. Claude guesses at statistics and cannot make charts.

Concept. Another server tool: Claude writes and runs Python in a sandbox container and sees the output. Great for calculations, data analysis and file processing. There is a separate execution-time charge beyond tokens, so check pricing.

```
# ch11_code.py
from client import client, MODEL

r = client.beta.messages.create(
    model=MODEL, max_tokens=1500,
    betas=['code-execution-2025-05-22'],
    tools=[{'type': 'code_execution_20250522', 'name': 'code_execution'}],
    messages=[{'role': 'user', 'content': 'Compute the mean and standard
deviation of 12, 15, 9, 22, 18 using code.'}],
)
for b in r.content:
    print(b.type)
    if b.type == 'text':
        print(b.text)
```

Watch out. If you get an error about the tool or header, the version strings changed: open the docs page and copy the current ones. The sandbox has no access to your computer or the internet.

Chapter 12: Files API

Status: Beta (header `files-api-2025-04-14` at time of writing; verify).

Problem. We keep re-uploading the same PDF.

Concept. Upload once, get a `file_id`, reuse it in any later request. You still pay input tokens for the content when Claude reads it, but not for re-sending bytes.

```
# ch12_files.py
from client import client, MODEL, text_of

f = client.beta.files.upload(file=('chapter.pdf', open('chapter.pdf', 'rb'),
'application/pdf'))
print('file id:', f.id)

r = client.beta.messages.create(
    model=MODEL, max_tokens=400,
    betas=['files-api-2025-04-14'],
    messages=[{'role': 'user', 'content': [
        {'type': 'document', 'source': {'type': 'file', 'file_id': f.id}},
        {'type': 'text', 'text': 'Summarize this in 3 bullets.'},
    ]}],
)
print(text_of(r))

for item in client.beta.files.list():
    print(item.id, item.filename)
# client.beta.files.delete(f.id)   # clean up when done
```

Watch out. Files live in your workspace until deleted. Do not upload anything private you would not want stored.

Act 4: Pagewise gets a library (RAG)

RAG means Retrieval-Augmented Generation: **find** the relevant pieces of your documents first, then **give only those** to Claude to answer. It is how you handle fifty books when one prompt cannot hold them (or would cost too much). Install one small library: `uv add rank-bm25`.

Chapter 13: RAG basics

Status: Stable (technique, not a single API feature).

Problem. We now have many documents.

Concept. Four steps: (1) **chunk** documents into pieces, (2) **index** them, (3) **retrieve** the top few pieces for a question, (4) **generate** an answer from just those pieces. Most tutorials use embeddings (vectors) for step 2 to 3. Anthropic does not offer an embedding

model of its own (it points to Voyage AI), so we start with **BM25**, a keyword ranking method that is free, local and a strong baseline. The pipeline is identical if you swap in embeddings later.

```
# rag.py
import re
from rank_bm25 import BM25Okapi
from client import client, MODEL, text_of


def chunk(text, size=800, overlap=150):
    out, i = [], 0
    while i < len(text):
        out.append(text[i:i + size])
        i += size - overlap
    return out


def tok(s):
    return re.findall(r'[a-z0-9]+', s.lower())


class Library:
    def __init__(self, text):
        self.chunks = chunk(text)
        self.bm25 = BM25Okapi([tok(c) for c in self.chunks])

    def search(self, query, k=4):
        scores = self.bm25.get_scores(tok(query))
        top = sorted(range(len(scores)), key=lambda i: scores[i],
reverse=True)[:k]
        return [(i, self.chunks[i]) for i in top]


def answer(lib, question):
    hits = lib.search(question)
    sources = '\n\n'.join(f'<source id={i}>{c}</source>' for i, c in hits)
    prompt = f'''{sources}

Question: {question}
Answer using only the sources. Cite source ids like [3]. If the sources do
not contain the answer, say you do not know.'''
    r = client.messages.create(model=MODEL, max_tokens=400,
```

```
                                messages=[{'role': 'user', 'content':
prompt}])
    return text_of(r)


if __name__ == '__main__':
    lib = Library(open('book.txt', encoding='utf-8').read())
    print(answer(lib, 'What does the book say about cell membranes?'))
```

Watch out. Chunk size is the main tuning knob: too small loses meaning, too large wastes tokens. Keyword search misses synonyms (the question says car, the text says automobile); that is the next chapter's problem. Always tell Claude to say so when the sources lack the answer.

Chapter 14: Better retrieval (contextual retrieval and reranking)

Status: Stable (technique). Contextual retrieval is an approach Anthropic published; I am describing it from memory, so check their write-up for current numbers.

Problem. A chunk like *It grew 3% that year* means nothing alone, so search cannot find it.

Concept. Contextual retrieval: before indexing, ask Claude to write one or two sentences that situate each chunk in its document, and prepend them. Prompt caching (Chapter 7) keeps this cheap because the whole document is the cached prefix. **Reranking**: retrieve more candidates (say 10) cheaply, then have a model pick the best 3.

```
# rag2.py
import re
from rank_bm25 import BM25Okapi
from client import client, MODEL, text_of
from rag import chunk, tok


def contextualize(doc, piece):
    r = client.messages.create(
        model=MODEL, max_tokens=120,
        system=[{'type': 'text', 'text': f'<document>\n{doc}\n</document>',
                 'cache_control': {'type': 'ephemeral'}}],
        messages=[{'role': 'user', 'content':
f'<chunk>\n{piece}\n</chunk>\nWrite a short context (1 to 2 sentences) that
```

```
situates this chunk within the whole document, to improve search. Answer with
the context only.'}],
    )
    return text_of(r)


class ContextLibrary:
    def __init__(self, doc, max_chunks=20):          # keep it small while
learning
        self.chunks = chunk(doc)[:max_chunks]
        self.ctx = [contextualize(doc, c) for c in self.chunks]
        self.bm25 = BM25Okapi([tok(x + ' ' + c) for x, c in zip(self.ctx,
self.chunks)])

    def search(self, query, k=10):
        scores = self.bm25.get_scores(tok(query))
        top = sorted(range(len(scores)), key=lambda i: scores[i],
reverse=True)[:k]
        return [(i, self.ctx[i] + ' ' + self.chunks[i]) for i in top]


def rerank(question, hits, keep=3):
    listing = '\n'.join(f'[{i}] {c[:300]}' for i, c in hits)
    r = client.messages.create(
        model=MODEL, max_tokens=30,
        messages=[{'role': 'user', 'content': f'Question:
{question}\nPassages:\n{listing}\nReturn the ids of the {keep} most useful
passages as comma-separated numbers only.'}],
    )
    ids = [int(x) for x in re.findall(r'[0-9]+', text_of(r))][:keep]
    return [(i, c) for i, c in hits if i in ids]
```

Run it. Build a `ContextLibrary`, search with a question that uses different words than the book, and compare against Chapter 13.

Watch out. This makes one model call per chunk when building the index, so cap `max_chunks` and run it once, saving the results to disk. Reranking adds a call per question.

Chapter 15: Agentic search

Status: Stable (technique).

Problem. Fixed RAG retrieves once. Hard questions need follow-up searches.

Concept. Instead of you retrieving, hand Claude a **search tool** and let it search, read, and search again. This is the *just-in-time* retrieval style used by modern coding agents, and it is the bridge from RAG to agents. Return short previews from search and let Claude open full chunks only when needed, which keeps context small.

```
# ch15_agentic_search.py
from client import text_of
from rag import Library
from toolkit import run_with_tools

lib = Library(open('book.txt', encoding='utf-8').read())


def search_notes(query):
    return '\n'.join(f'[{i}] {c[:150]}...' for i, c in lib.search(query,
k=4))


def read_chunk(chunk_id):
    return lib.chunks[int(chunk_id)]


tools = [
    {'name': 'search_notes', 'description': 'Keyword search over the book.
Returns short previews with ids.',
     'input_schema': {'type': 'object', 'properties': {'query': {'type':
'string'}}, 'required': ['query']}},
    {'name': 'read_chunk', 'description': 'Read the full text of one chunk by
id, after searching.',
     'input_schema': {'type': 'object', 'properties': {'chunk_id': {'type':
'integer'}}, 'required': ['chunk_id']}},
]
system = 'Search before answering. You may search several times. Cite chunk
ids like [3]. If you cannot find it, say so.'
msgs = [{'role': 'user', 'content': 'Compare how the book describes osmosis
and diffusion.'}]
final = run_with_tools(msgs, tools, {'search_notes': search_notes,
'read_chunk': read_chunk},
                       system=system, max_turns=6)
print(text_of(final))
```

Watch out. More flexible means more calls and more cost. Use fixed RAG for simple lookups and agentic search when questions need exploration. Always cap `max_turns`.

Act 5: Pagewise stays sharp (context engineering)

Prompt engineering is writing good instructions. **Context engineering** is deciding everything that sits in the model's context at each step: system prompt, tools, retrieved data, message history, notes. Anthropic's framing is that context is a finite resource with diminishing returns, so the goal is the smallest set of high-signal tokens that gets the job done. Bigger windows do not remove this problem; more irrelevant text can make answers worse and bills bigger.

Chapter 16: The attention budget

Status: Stable (principles plus a measuring script).

Problem. Pagewise is slow and expensive because we dump everything into the prompt.

Concept. Four parts of context to manage: the **system prompt** (specific enough to guide, flexible enough not to be brittle), the **tools** (few, clear, non-overlapping), **examples** (a few good ones beat a long list of rules) and **message history** (prune it). Three habits: load data *just in time* instead of up front, reveal detail *progressively* (previews first, full text on request), and *measure* before you optimize.

```
# ch16_context_budget.py
from client import client, MODEL, text_of
from rag import Library

book = open('book.txt', encoding='utf-8').read()
lib = Library(book)
q = 'What is osmosis?'
focused = '\n\n'.join(c for _, c in lib.search(q, k=3))


def ask_with(material):
    r = client.messages.create(model=MODEL, max_tokens=200,
        messages=[{'role': 'user', 'content':
f'<material>\n{material}\n</material>\n\n{q} Answer in 2 sentences.'}])
    return text_of(r), r.usage.input_tokens
```

```
for name, material in [('everything', book), ('3 focused chunks', focused)]:
    answer, tokens = ask_with(material)
    print(f'{name}: {tokens} input tokens')
    print(answer)
    print('-' * 40)


def audit(system, tools, messages):
    # where does your context budget go?
    total = client.messages.count_tokens(model=MODEL, system=system,
tools=tools, messages=messages).input_tokens
    no_tools = client.messages.count_tokens(model=MODEL, system=system,
messages=messages).input_tokens
    print('total:', total, '| tool definitions cost about:', total -
no_tools)
```

Watch out. Compare answer *quality*, not only token counts. Tool definitions are paid for on every single call, which is one reason to keep the tool list short.

Chapter 17: Compaction and context editing

Status: DIY version Stable. Native context editing and server-side compaction are Beta (verify the context management docs).

Problem. A long study session fills the window.

Concept. Compaction: when history gets big, have Claude summarize the old part and continue from the summary plus the last few turns. **Context editing**: clear stale tool results, which are usually the biggest and least useful part. Build the simple version yourself first so you understand it.

```
# compact.py
from client import client, MODEL, text_of


def render(messages):
    lines = []
    for m in messages:
        c = m['content']
        if isinstance(c, str):
```

```
            lines.append(m['role'] + ': ' + c)
            continue
        for b in c:
            if isinstance(b, dict):
                if b.get('type') == 'tool_result':
                    lines.append('tool result: ' + str(b.get('content'))
[:300])
                elif b.get('text'):
                    lines.append(m['role'] + ': ' + b['text'])
            elif getattr(b, 'text', None):
                lines.append(m['role'] + ': ' + b.text)
    return '\n'.join(lines)


def split_point(messages, keep_last=4):
    # cut only where a plain user message starts a turn, so tool_use and
tool_result stay paired
    for i in range(len(messages) - keep_last, 0, -1):
        if messages[i]['role'] == 'user' and isinstance(messages[i]
['content'], str):
            return i
    return 0


def compact(messages, keep_last=4):
    i = split_point(messages, keep_last)
    if i <= 0:
        return messages
    old, recent = messages[:i], messages[i:]
    r = client.messages.create(model=MODEL, max_tokens=500, messages=
[{'role': 'user', 'content':
        'Summarize this conversation so work can continue. Keep goals,
decisions, key facts, files touched and open tasks. Be brief.\n\n' +
render(old)}])
    head = [{'role': 'user', 'content': '<summary of earlier conversation>\n'
+ text_of(r) + '\n</summary>'},
            {'role': 'assistant', 'content': 'Understood. Continuing from the
summary.'}]
    return head + recent
```

```
def maybe_compact(messages, limit_tokens=20000):
    n = client.messages.count_tokens(model=MODEL,
messages=messages).input_tokens
    return compact(messages) if n > limit_tokens else messages
```

Native option (Beta). Context editing can automatically clear old tool results. The shape at the time of writing is below; copy current strings from the docs.

```
r = client.beta.messages.create(
    model=MODEL, max_tokens=1000,
    betas=['context-management-2025-06-27'],
    context_management={'edits': [{'type': 'clear_tool_uses_20250919'}]},
    tools=tools, messages=messages,
)
```

Watch out. Compaction is lossy: you decide what survives, and later steps may need a detail you dropped. Put important facts (goals, constraints) in the summary prompt explicitly. Never cut between a `tool_use` and its `tool_result`.

Chapter 18: Memory and notes outside the window

Status: DIY version Stable. The official memory tool is Beta.

Problem. Tomorrow, Pagewise has forgotten who you are.

Concept. Give the agent **notes it writes itself**, stored in files outside the context window, and have it read them when a session starts. The context window is the working desk; files are the filing cabinet. Anthropic also ships a **memory tool** (`memory_20250818` , a client-side tool where you implement file operations like view, create and delete); the DIY version below teaches the same idea in a few lines.

```
# memory.py
import os

MEM = 'memory'
os.makedirs(MEM, exist_ok=True)


def _path(name):
    safe = ''.join(ch for ch in name if ch.isalnum() or ch in '-_')
```

```
    return os.path.join(MEM, safe + '.md')


def write_note(name, text):
    open(_path(name), 'w', encoding='utf-8').write(text)
    return 'saved ' + name


def read_note(name):
    return open(_path(name), encoding='utf-8').read()


def list_notes():
    return ', '.join(sorted(f[:-3] for f in os.listdir(MEM) if
f.endswith('.md'))) or 'no notes yet'


MEMORY_TOOLS = [
    {'name': 'list_notes', 'description': 'List saved memory notes.',
     'input_schema': {'type': 'object', 'properties': {}}},
    {'name': 'read_note', 'description': 'Read one memory note by name.',
     'input_schema': {'type': 'object', 'properties': {'name': {'type':
'string'}}, 'required': ['name']}},
    {'name': 'write_note', 'description': 'Save or overwrite a durable note
about the student (goals, weak topics, preferences).',
     'input_schema': {'type': 'object', 'properties': {'name': {'type':
'string'}, 'text': {'type': 'string'}}, 'required': ['name', 'text']}},
]
MEMORY_HANDLERS = {'list_notes': list_notes, 'read_note': read_note,
'write_note': write_note}
```

```
# ch18_memory.py
from client import text_of
from memory import MEMORY_TOOLS, MEMORY_HANDLERS
from toolkit import run_with_tools

system = 'You are Pagewise. At the start of a session call list_notes and
read relevant notes. Save durable facts about the student with write_note.'

# session 1
m1 = [{'role': 'user', 'content': 'I am Asha, my exam is on genetics in
March, and I struggle with meiosis. Remember that.'}]
print(text_of(run_with_tools(m1, MEMORY_TOOLS, MEMORY_HANDLERS,
system=system)))

# session 2: brand new message list, only the files carry over
m2 = [{'role': 'user', 'content': 'What should I study this week?'}]
print(text_of(run_with_tools(m2, MEMORY_TOOLS, MEMORY_HANDLERS,
system=system)))
```

Watch out. Memory is a place for prompt injection and stale facts: store only what is useful, let the student view and delete notes, and never store secrets. Keep notes short; reading ten pages of notes every session defeats the purpose.

Chapter 19: Tool design, tool search, programmatic tool calling

Status: design principles Stable. Tool search and programmatic tool calling are Beta (verify in the tool use docs).

Problem. With many tools Claude picks the wrong one, and the definitions eat the budget.

Concept. Design tools the way you would design an interface for a new colleague: a clear name, a description that says *when* to use it and when not to, few parameters, no overlap with other tools, short high-signal results, and error messages that tell Claude what to try next. For large tool sets, **tool search** lets Claude load tool definitions only when needed, and **programmatic tool calling** lets Claude call tools from code to process many results without pushing all of them through its context. Both are Beta features; read their docs pages when you reach that scale.

```
# ch19_tool_design.py
bad = {'name': 'search', 'description': 'searches',
       'input_schema': {'type': 'object', 'properties': {'q': {'type':
'string'}}}}

good = {
    'name': 'search_notes',
    'description': ('Keyword search over the student course notes. Use for
questions about course content. '
                    'Returns previews with ids; call read_chunk for full
text. Not for current events.'),
    'input_schema': {
        'type': 'object',
        'properties': {
            'query': {'type': 'string', 'description': 'two to five keywords,
not a full sentence'},
            'limit': {'type': 'integer', 'description': 'max results, default
4, max 8'},
            'detail': {'type': 'string', 'enum': ['concise', 'detailed'],
'description': 'concise = 150 chars per hit'},
        },
        'required': ['query'],
    },
}


def search_handler(lib, query, limit=4, detail='concise'):
    hits = lib.search(query, k=min(limit, 8))
    if not hits:
        return 'No results. Try fewer or different keywords.'          #
guide the next step
    width = 150 if detail == 'concise' else 600
    out = '\n'.join(f'[{i}] {c[:width]}' for i, c in hits)
    return out[:3000]                                                   #
never return unbounded text
```

Watch out. When your own agent keeps misusing a tool, fix the description before you fix the prompt. Read a few real transcripts of tool calls; the mistakes are usually obvious.

Act 6: Pagewise becomes an agent

Chapter 20: The agent loop, by hand

Status: Stable (it is plain Python plus the Messages API).

Problem. Pagewise should finish a multi-step job on its own: read notes, write a summary file, check it.

Concept. An **agent** is a model using tools in a loop until the job is done. Four ingredients: the **loop**, the **tools**, **stop conditions** (finished, out of turns, out of money, stuck) and **observability** (print what it does). Everything fancier is built from these. Claude sometimes repeats the same failing call, so we add a cheap stuck detector.

```
# agent.py
import json
import os
from client import client, MODEL, text_of

WORKSPACE = 'workspace'
os.makedirs(WORKSPACE, exist_ok=True)


def safe(path):
    full = os.path.abspath(os.path.join(WORKSPACE, path))
    if not full.startswith(os.path.abspath(WORKSPACE) + os.sep):
        raise ValueError('path is outside the workspace')
    return full


def read_file(path):
    return open(safe(path), encoding='utf-8').read()


def write_file(path, content):
    open(safe(path), 'w', encoding='utf-8').write(content)
    return 'wrote ' + path


def list_files():
    return ', '.join(os.listdir(WORKSPACE)) or 'empty'
```

```
TOOLS = [
    {'name': 'list_files', 'description': 'List files in the workspace.',
'input_schema': {'type': 'object', 'properties': {}}},
    {'name': 'read_file', 'description': 'Read a text file from the
workspace.',
     'input_schema': {'type': 'object', 'properties': {'path': {'type':
'string'}}, 'required': ['path']}},
    {'name': 'write_file', 'description': 'Create or overwrite a text file in
the workspace.',
     'input_schema': {'type': 'object', 'properties': {'path': {'type':
'string'}, 'content': {'type': 'string'}}, 'required': ['path', 'content']}},
]
HANDLERS = {'list_files': list_files, 'read_file': read_file, 'write_file':
write_file}


def run_agent(goal, system, tools=TOOLS, handlers=HANDLERS, max_turns=10,
max_cost_usd=0.10):
    messages = [{'role': 'user', 'content': goal}]
    spent, seen = 0.0, set()
    for turn in range(1, max_turns + 1):
        resp = client.messages.create(model=MODEL, max_tokens=1500,
system=system, tools=tools, messages=messages)
        spent += resp.usage.input_tokens / 1e6 * 1.0 +
resp.usage.output_tokens / 1e6 * 5.0   # Haiku prices; update
        messages.append({'role': 'assistant', 'content': resp.content})
        if resp.stop_reason != 'tool_use':
            return {'status': 'done', 'answer': text_of(resp), 'turns': turn,
'cost': round(spent, 4)}
        results = []
        for b in resp.content:
            if b.type != 'tool_use':
                continue
            key = (b.name, json.dumps(b.input, sort_keys=True))
            if key in seen:
                out, err = 'You already made this exact call. Try something
different or finish.', True
            else:
                seen.add(key)
                try:
                    out, err = handlers[b.name](**b.input), False
```

```
                except Exception as e:
                    out, err = f'Error: {e}', True
            print(f'turn {turn}: {b.name}({b.input})')
            item = {'type': 'tool_result', 'tool_use_id': b.id, 'content':
str(out)[:4000]}
            if err:
                item['is_error'] = True
            results.append(item)
        messages.append({'role': 'user', 'content': results})
        if spent > max_cost_usd:
            return {'status': 'over_budget', 'turns': turn, 'cost':
round(spent, 4)}
    return {'status': 'max_turns', 'turns': max_turns, 'cost': round(spent,
4)}
```

```
# ch20_run.py
from agent import run_agent, write_file

write_file('notes.txt', 'Mitosis makes two identical cells. Meiosis makes
four different cells with half the chromosomes. Meiosis involves crossing
over.')
result = run_agent(
    goal='Read notes.txt, then write summary.md with 3 bullet points, then
tell me you are done.',
    system='You are Pagewise. Work only inside the workspace. Be efficient.',
)
print(result)
```

Watch out. Always return a status, not just an answer, so you know *why* it stopped. A path check like `safe()` is the minimum when an agent can touch files. Print each step while learning; you will spot wasted turns immediately.

Chapter 21: Workflows versus agents

Status: Stable (patterns).

Problem. Not every job needs a free-roaming agent, and agents cost more.

Concept. Use the simplest thing that works. **Workflows** are fixed paths *you* script (predictable, cheaper); **agents** decide their own steps (flexible, costlier). The five classic workflow patterns: prompt chaining, routing, parallelization, orchestrator-workers,

evaluator-optimizer. This is the pattern vocabulary from Anthropic's Building Effective Agents guide.

```
# patterns.py
from concurrent.futures import ThreadPoolExecutor
from client import client, MODEL, text_of


def ask(prompt, system=None, max_tokens=500):
    kw = dict(model=MODEL, max_tokens=max_tokens, messages=[{'role': 'user',
'content': prompt}])
    if system:
        kw['system'] = system
    return text_of(client.messages.create(**kw))


# 1. Prompt chaining: each step feeds the next
def chain(topic):
    outline = ask(f'Write a 3 point outline about {topic}.')
    return ask(f'Write a short lesson following this outline:\n{outline}')


# 2. Routing: classify first, then use the right specialist
SPECIALISTS = {
    'definition': 'Give a one sentence definition.',
    'quiz': 'Write one multiple choice question with the answer.',
    'summary': 'Summarize in 3 bullets.',
}


def route(request):
    label = ask(f'Classify this request as definition, quiz or summary. Reply
with one word.\n{request}', max_tokens=5).strip().lower()
    return ask(request, system=SPECIALISTS.get(label,
SPECIALISTS['summary']))


# 3. Parallelization: independent calls at the same time
def parallel(topics):
    with ThreadPoolExecutor(max_workers=4) as ex:
        return list(ex.map(lambda t: ask(f'One line summary of: {t}',
max_tokens=80), topics))
```

```
# 4. Orchestrator-workers: one call plans, others execute, one combines
def orchestrate(topic):
    plan = ask(f'List 3 subtopics of {topic}, one per line, no numbering.',
max_tokens=80).splitlines()
    plan = [p.strip() for p in plan if p.strip()][:3]
    with ThreadPoolExecutor(max_workers=3) as ex:
        parts = list(ex.map(lambda s: ask(f'Explain {s} in 2 sentences.',
max_tokens=150), plan))
    return ask('Combine these into one short study note:\n' +
'\n'.join(parts))


# 5. Evaluator-optimizer: write, critique, revise until it passes or tries
run out
def refine(task, rounds=3):
    draft = ask(task)
    for _ in range(rounds):
        critique = ask(f'Task: {task}\nDraft: {draft}\nIf the draft fully
meets the task reply only PASS. Otherwise give 2 specific fixes.',
max_tokens=150)
        if critique.strip().upper().startswith('PASS'):
            break
        draft = ask(f'Task: {task}\nDraft: {draft}\nFixes: {critique}\nWrite
the improved draft.')
    return draft
```

Decision guide. One good prompt first. If steps are known in advance, use a workflow. If the steps depend on what is discovered along the way, use an agent. Add complexity only when a measurement (Chapter 27) shows you need it.

Watch out. Parallel calls count against rate limits. The evaluator-optimizer loop can burn money; keep `rounds` small.

Chapter 22: Agent Skills and MCP

Status: Beta. Header names, tool versions and field names changed several times; copy current ones from the Skills and MCP connector docs.

Problem. Pagewise needs reusable know-how (how to format a study pack) and access to outside systems (your Notion, your Drive).

Concept. A **Skill** is a folder with a `SKILL.md` (a name, a description that says when to use it, and instructions) plus optional scripts. Claude reads the short description first and loads the full instructions only when relevant, which is progressive disclosure again. **MCP** (Model Context Protocol) is a standard way to expose tools and data from outside systems; the API's **MCP connector** lets Claude reach a remote MCP server without you writing the client code.

A skill file you can use with Claude Code or the Agent SDK (put it in a `skills/study-notes/` folder):

```
---
name: study-notes
description: Use when the student asks for study notes, a cheat sheet or a
revision summary of course material.
---

# Study notes format
1. Start with a 1 sentence overview.
2. List key terms in bold with a short definition.
3. End with 3 self-test questions and answers.
```

The API shapes at the time of writing (verify before use):

```
# MCP connector (Beta): Claude calls a remote MCP server for you
r = client.beta.messages.create(
    model=MODEL, max_tokens=800,
    betas=['mcp-client-2025-04-04'],
    mcp_servers=[{'type': 'url', 'url': 'https://YOUR-MCP-SERVER/mcp',
'name': 'notes'}],
    messages=[{'role': 'user', 'content': 'Which notes did I add this
week?'}],
)

# Pre-built Skills (Beta): Anthropic's document skills run inside code
execution
r = client.beta.messages.create(
    model=MODEL, max_tokens=2000,
    betas=['code-execution-2025-08-25', 'skills-2025-10-02'],
    container={'skills': [{'type': 'anthropic', 'skill_id': 'pptx',
'version': 'latest'}]},
    tools=[{'type': 'code_execution_20250825', 'name': 'code_execution'}],
    messages=[{'role': 'user', 'content': 'Make a 3 slide deck about
mitosis.'}],
)
```

Watch out. A newer MCP connector version moves server settings into the `tools` list instead of `mcp_servers`; if the first form errors, read the current docs. Only connect to MCP servers you trust, because their tool descriptions and results go into Claude's context.

Chapter 23: The Claude Agent SDK

Status: Stable package, fast-moving details. Check the Agent SDK quickstart.

Problem. We hand-built a loop in Chapter 20. Do we have to rebuild file tools, permissions and context handling every time?

Concept. The **Claude Agent SDK** is the runtime that powers Claude Code, packaged as a Python and TypeScript library. It gives you the agent loop, built-in tools (Read, Write, Edit, Bash, Glob, Grep, web tools), permissions, and context management. You describe the job; it runs the loop. It renamed from Claude Code SDK in September 2025. Use it when you want a capable coding or file agent quickly; use your own loop (Chapter 20) when you want full control and the smallest cost.

```
uv add claude-agent-sdk
# if it complains about a missing runtime, install Claude Code: npm install -
g @anthropic-ai/claude-code
```

```
# ch23_sdk.py
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions, AssistantMessage,
TextBlock, ResultMessage


async def main():
    options = ClaudeAgentOptions(
        system_prompt='You are Pagewise. Be concise.',
        allowed_tools=['Read', 'Write', 'Glob'],
        permission_mode='acceptEdits',          # auto-approve file edits;
keep tools few
        max_turns=6,                            # always cap the loop
        model='claude-haiku-4-5-20251001',      # set the model yourself to
control cost
        cwd='workspace',                        # the folder it may work in
(must exist)
    )
    async for message in query(prompt='Read notes.txt and write summary.md
with 3 bullets.', options=options):
        if isinstance(message, AssistantMessage):
            for block in message.content:
                if isinstance(block, TextBlock):
                    print(block.text)
        elif isinstance(message, ResultMessage):
            print('cost:', getattr(message, 'total_cost_usd', None),
'turns:', getattr(message, 'num_turns', None))


asyncio.run(main())
```

Custom tools run inside your own process through an in-process MCP server:

```
from claude_agent_sdk import tool, create_sdk_mcp_server, ClaudeAgentOptions


@tool('lookup_term', 'Look up a term in the course glossary', {'term': str})
async def lookup_term(args):
    return {'content': [{'type': 'text', 'text': 'Mitosis: cell division
making two identical cells.'}]}


server = create_sdk_mcp_server(name='pagewise', version='1.0.0', tools=
[lookup_term])
options = ClaudeAgentOptions(
    mcp_servers={'pagewise': server},
    allowed_tools=['mcp__pagewise__lookup_term'],   # naming rule:
mcp__server__tool
)
```

Watch out. The SDK carries more built-in instructions and tools than your own loop, so each call can cost more. Always set `model`, `max_turns` and the tool list. Use API-key authentication; Anthropic does not allow third-party apps to offer claude.ai logins. Older SDK versions may not accept the newest models: upgrade if you see a model error.

Chapter 24: Harness engineering (agents that survive many sessions)

Status: Stable (pattern). Based on Anthropic's engineering write-up on long-running agents; the code below is my own cheap version.

Problem. A job is too big for one context window, and each new session starts with amnesia.

Concept. A **harness** is everything around the model that makes it reliable: the loop, tools, context handling, memory and checks. For long work Anthropic's pattern is: an **initializer** runs once and writes a plan (a task list with every item marked not passing, a progress file, a way to start the project). Every later session starts with a fresh context, reads the progress file and task list, does **one** task, **verifies** it, records the result, and leaves the workspace clean. State lives in files, not in the chat. Our version builds a study pack (a summary and flashcards) for every chapter file. Put `.txt` chapter files in a folder named `chapters/`.

```
# harness.py
import json
import os
from client import text_of
from toolkit import run_with_tools

STATE, OUT = 'state', 'out'
os.makedirs(STATE, exist_ok=True)
os.makedirs(OUT, exist_ok=True)
TASKS = os.path.join(STATE, 'tasks.json')
PROGRESS = os.path.join(STATE, 'progress.md')


def load_tasks():
    return json.load(open(TASKS))


def save_tasks(tasks):
    json.dump(tasks, open(TASKS, 'w'), indent=2)


def log(*parts):
    open(PROGRESS, 'a', encoding='utf-8').write(' | '.join(parts) + '\n')


def init(chapter_files):
    # THE INITIALIZER: runs once and writes the plan
    tasks = [{'id': f'{name}-{kind}', 'chapter': name, 'kind': kind,
'passes': False, 'attempts': 0}
             for name in chapter_files for kind in ('summary', 'flashcards')]
    save_tasks(tasks)
    log('init', f'{len(tasks)} tasks created')


def save_output(filename, content):
    open(os.path.join(OUT, os.path.basename(filename)), 'w', encoding='utf-
8').write(content)
    return 'saved ' + filename


SAVE_TOOL = [{'name': 'save_output', 'description': 'Save the finished output
file.',
              'input_schema': {'type': 'object',
```

```
                                'properties': {'filename': {'type': 'string'},
'content': {'type': 'string'}},
                                'required': ['filename', 'content']}}]


def verify(task):
    # check the ARTIFACT, never the model's claim that it is done
    ext = '.md' if task['kind'] == 'summary' else '.json'
    path = os.path.join(OUT, task['id'] + ext)
    if not os.path.exists(path):
        return False, 'output file missing'
    if task['kind'] == 'summary':
        n = sum(1 for line in open(path, encoding='utf-8') if
line.strip().startswith('-'))
        return n >= 3, f'{n} bullets'
    try:
        cards = json.load(open(path, encoding='utf-8'))
    except Exception as e:
        return False, f'invalid json: {e}'
    return isinstance(cards, list) and len(cards) >= 5, f'{len(cards)} cards'


def run_session():
    tasks = load_tasks()
    task = next((t for t in tasks if not t['passes'] and t['attempts'] < 3),
None)
    if task is None:
        return 'idle'
    tid, kind = task['id'], task['kind']
    tail = open(PROGRESS, encoding='utf-8').read()[-1500:] if
os.path.exists(PROGRESS) else ''
    text = open(os.path.join('chapters', task['chapter']), encoding='utf-
8').read()[:6000]
    if kind == 'summary':
        spec = f'file name {tid}.md: 3 to 5 bullet lines, each starting with
a dash'
    else:
        spec = f'file name {tid}.json: a JSON list of at least 5 objects with
question and answer keys'
    goal =
f'<progress_log>\n{tail}\n</progress_log>\n<chapter>\n{text}\n</chapter>\nTas
k {tid}: create {spec}. Save it with save_output.'
    system = 'You are one worker session in a harness. Do only this one task.
```

```
The harness verifies your file, so do not claim success.'
    task['attempts'] += 1
    try:
        run_with_tools([{'role': 'user', 'content': goal}], SAVE_TOOL,
{'save_output': save_output},
                       system=system, max_turns=4, max_tokens=1500)
    except RuntimeError as e:
        log(tid, 'error', str(e))
    ok, note = verify(task)
    task['passes'] = ok
    save_tasks(tasks)
    log(tid, 'PASS' if ok else 'FAIL', note, f'attempt {task["attempts"]}')
    return ('pass:' if ok else 'fail:') + tid
```

```
# ch24_run.py
import os
from harness import init, run_session

if not os.path.exists('state/tasks.json'):
    init(sorted(f for f in os.listdir('chapters') if f.endswith('.txt')))
print(run_session())      # one session = one task; run the file again for
the next
```

Run it several times and read `state/progress.md` after each run. Delete the `state/` folder to start over.

Why this works. (1) The plan and the progress live on disk, so any fresh session can continue. (2) One task per session keeps context small. (3) Deterministic verification stops the model from declaring victory early. (4) Bounded retries stop endless loops. Anthropic's coding version adds a feature list in JSON with a pass flag per feature, a startup script, and a git commit after each finished feature.

Watch out. Harness assumptions about what the model cannot do go stale as models improve; revisit them. Do not let sessions edit the task list except for the pass flag.

Chapter 25: Loop engineering (making it run by itself)

Status: Stable (design practice). Note: this term is newer and community-driven, not an official Anthropic product feature. The code is my own.

Problem. Someone has to keep typing *run it again*.

Concept. Loop engineering is designing the outer cycle that decides what to do next, runs a session, checks the result, records it and decides whether to continue, so that work moves toward a goal without a person driving each turn. It sits above the harness: prompts shape one call, context shapes what the model sees, the harness shapes one session, the loop shapes the whole run. Every loop needs these eight decisions:

Decision	Pagewise answer
Trigger	You run it, or a nightly schedule (cron or Task Scheduler)
Goal (done means)	Every task in `tasks.json` has `passes: true`
Unit of work	One task per session
Verification	Deterministic file checks, not the model's opinion
State	`tasks.json`, `progress.md`, `loop.jsonl`
Budget	Max sessions and max dollars
Stop conditions	Done, over budget, stuck on the same failure, kill-switch file
Escalation	Print a clear message for a human when stuck

```
# loop.py
import json
import os
import time
from harness import init, run_session, load_tasks
from toolkit import cost_usd

MAX_SESSIONS = 12
MAX_COST_USD = 0.50
KILL_SWITCH = 'state/STOP'          # create this file to stop the loop
politely


def event(e):
    open('state/loop.jsonl', 'a', encoding='utf-8').write(json.dumps(e) +
'\n')
```

```
def run_loop():
    if not os.path.exists('state/tasks.json'):
        init(sorted(f for f in os.listdir('chapters') if f.endswith('.txt')))
    last_fail, repeats = None, 0
    for n in range(1, MAX_SESSIONS + 1):
        if os.path.exists(KILL_SWITCH):
            return 'stopped: kill switch'
        status = run_session()
        spent = cost_usd()
        event({'n': n, 'status': status, 'cost': round(spent, 4), 'time':
time.time()})
        print(n, status, f'${spent:.3f}')
        if status == 'idle':
            left = [t['id'] for t in load_tasks() if not t['passes']]
            return 'finished' if not left else 'needs a human, still failing:
' + ', '.join(left)
        if spent >= MAX_COST_USD:
            return 'stopped: budget reached'
        if status.startswith('fail:'):
            repeats = repeats + 1 if status == last_fail else 1
            last_fail = status
            if repeats >= 3:
                return 'stopped: stuck on ' + status
        else:
            last_fail, repeats = None, 0
    return 'stopped: session limit'


if __name__ == '__main__':
    print(run_loop())
```

Watch out. A loop made of badly prompted sessions just fails faster and costs more. Always have a budget, a stuck detector and a kill switch. Give a looping agent the least permission it needs. Anthropic's Claude Code has features for repeating and scheduled work, and community articles describe commands such as `/loop`; I have not verified their current names, so read the Claude Code docs for those.

Chapter 26: Sub-agents and a separate evaluator

Status: Stable (pattern).

Problem. One agent reading ten files fills its own context, and agents tend to rate their own work too generously.

Concept. A **sub-agent** is a fresh agent with its own context given one narrow job. It may read a lot, but it hands back only a short report, so the orchestrator's context stays clean. An **evaluator** is a separate model call with a strict rubric that judges the work; keeping it separate from the writer avoids self-flattery. The cost is real: multi-agent systems can use many times more tokens than one chat, so use them only when a task is truly parallel or too big for one context.

```
# multiagent.py
from client import client, MODEL, text_of
from toolkit import run_with_tools
from agent import TOOLS, HANDLERS

READ_TOOLS = [t for t in TOOLS if t['name'] in ('list_files', 'read_file')]
READ_HANDLERS = {k: v for k, v in HANDLERS.items() if k in ('list_files',
'read_file')}


def sub_agent(task):
    # fresh context; returns a SHORT report, not everything it read
    msgs = [{'role': 'user', 'content': task}]
    final = run_with_tools(msgs, READ_TOOLS, READ_HANDLERS,
                           system='You are a research sub-agent. Reply with a
report of at most 120 words.',
                           max_turns=5, max_tokens=600)
    return text_of(final)


DELEGATE = [{'name': 'delegate', 'description': 'Give one independent sub-
task to a sub-agent that can read workspace files. Returns a short report.',
             'input_schema': {'type': 'object', 'properties': {'task':
{'type': 'string'}}, 'required': ['task']}}]
```

```
def orchestrate(goal):
    msgs = [{'role': 'user', 'content': goal}]
    final = run_with_tools(msgs, DELEGATE, {'delegate': sub_agent},
                           system='Split the goal into independent sub-tasks,
delegate each, then write the final answer from the reports.',
                           max_turns=6, max_tokens=800)
    return text_of(final)


GRADE = [{'name': 'grade', 'description': 'Grade the work against the
rubric.',
          'input_schema': {'type': 'object',
                           'properties': {'passed': {'type': 'boolean'},
'feedback': {'type': 'string'}},
                           'required': ['passed', 'feedback']}}]


def evaluate(task, work, rubric):
    r = client.messages.create(
        model=MODEL, max_tokens=300, tools=GRADE, tool_choice={'type':
'tool', 'name': 'grade'},
        system='You are a strict grader. Judge only against the rubric. Do
not be generous.',
        messages=[{'role': 'user', 'content': f'Task: {task}\nRubric:
{rubric}\nWork:\n{work}'}],
    )
    return next(b.input for b in r.content if b.type == 'tool_use')
```

Run it. Put two or three `.txt` files in `workspace/`, then call `orchestrate('Compare what each file says about meiosis.')`. Print the sub-agent calls to see the context isolation working.

Watch out. Sub-agents share nothing automatically; pass what they need in the task text. Calibrate the evaluator with a few examples of good and bad work, and check its grades by hand at first.

Chapter 27: Evals and tracing

Status: Stable (technique).

Problem. Is Pagewise actually good, or did your last change quietly break something?

Concept. An **eval** is a set of test cases with automatic grading, run after every change. Start with 10 to 20 cases drawn from real failures. Grade the *outcome* rather than the exact steps. Use deterministic checks first (does the answer contain the key term, is the JSON valid), and an LLM judge with a narrow rubric for what code cannot check. **Tracing** logs every call (tokens, time, stop reason) so you can see why something failed.

```
# evals.py
import json
import time
from client import client, MODEL
from rag import Library, answer
from multiagent import evaluate

lib = Library(open('book.txt', encoding='utf-8').read())

CASES = [
    {'q': 'What does the book say about cell membranes?', 'must_include':
['membrane'],
     'rubric': 'States that membranes control what enters and leaves the
cell.'},
    {'q': 'Who won the 1998 football World Cup?', 'must_include': ['know'],
     'rubric': 'Admits the book does not contain this information and does
not guess.'},
]


def run_evals():
    passed = 0
    for case in CASES:
        out = answer(lib, case['q'])
        det_ok = all(w.lower() in out.lower() for w in case['must_include'])
        verdict = evaluate(case['q'], out, case['rubric'])
        ok = det_ok and verdict['passed']
        passed += ok
        print('PASS' if ok else 'FAIL', '|', case['q'], '|',
verdict['feedback'][:80])
    print(f'{passed}/{len(CASES)} passed')


def traced_create(**kw):
    t0 = time.time()
    r = client.messages.create(**kw)
```

```
    row = {'time': t0, 'ms': int((time.time() - t0) * 1000), 'model':
kw.get('model'),
           'in': r.usage.input_tokens, 'out': r.usage.output_tokens, 'stop':
r.stop_reason}
    open('state/trace.jsonl', 'a', encoding='utf-8').write(json.dumps(row) +
'\n')
    return r


if __name__ == '__main__':
    run_evals()
```

Watch out. An LLM judge can be wrong, so read its explanations until you trust it. Keep every failure you find as a permanent test case. Re-run evals after any change to prompts, tools or models.

Epilogue: Going to production

Errors and retries. The SDK retries transient failures automatically; tune it and handle what remains:

```
import anthropic

client = anthropic.Anthropic(max_retries=4, timeout=60)
try:
    client.messages.create(model=MODEL, max_tokens=100, messages=[{'role':
'user', 'content': 'hi'}])
except anthropic.RateLimitError:
    print('slow down and retry later')
except anthropic.APIConnectionError:
    print('network problem')
except anthropic.APIStatusError as e:
    print(e.status_code, e.message)
```

Production checklist.

- Secrets in a secret manager, never in code; one key per app so you can revoke one.
- Budgets and alerts in Console; the Usage and Cost API and Admin API exist for teams.
- Treat tool results and web content as untrusted (prompt injection). Least-privilege tools, sandboxed paths.
- Log every call; keep an eval suite; pin model versions and re-test before upgrading.

- Ask about data retention and Zero Data Retention if you handle sensitive data.
- Consider **Managed Agents**, Anthropic's hosted service for long-running agents, when you want them to run your harness for you (concept only here; it is not worth spending the learning budget on).

Appendix A: Cheat sheet

You want	Use
Cap reply length	`max_tokens`
Set role and rules	`system`
Force JSON shape	`tools` + `tool_choice={'type': 'tool', 'name': ...}`
Stream output	`client.messages.stream(...)`
Reason harder	`thinking={'type': 'enabled', 'budget_tokens': N}` (older style)
Source quotes	`document` block with `citations: {enabled: True}`
Size a prompt	`client.messages.count_tokens(...)`
Reuse a big prefix	`cache_control: {'type': 'ephemeral'}`
Cheap bulk	`client.messages.batches.create(...)`
Call your function	tools, then `tool_result` with the same `tool_use_id`
Live info	web search server tool
Run Python	code execution server tool
Reuse uploads	Files API (`client.beta.files`)
Beta feature	`client.beta.messages.create(betas=[...])`

Appendix B: Troubleshooting

Symptom	Likely cause
Authentication error	`ANTHROPIC_API_KEY` not set in this terminal
400 about roles	Messages do not alternate, or a `tool_use` has no matching `tool_result`
Cut-off answer	`stop_reason` is `max_tokens`; raise it
Cache numbers all zero	Prompt shorter than the minimum, or prefix changed
Beta or tool error	Version string or header outdated; copy from the docs page
Agent loops forever	No `max_turns`, vague goal, or no stuck detector
Bill higher than expected	Large context re-sent each call, or too many loop steps; check Console Usage

Appendix C: Reading list (Anthropic engineering posts)

- Building effective agents
- Effective context engineering for AI agents
- Effective harnesses for long-running agents
- How we built our multi-agent research system
- Writing tools for AI agents
- Code execution with MCP: building more efficient agents
- Demystifying evals for AI agents
- Scaling Managed Agents: decoupling the brain from the hands

Appendix D: What I could not verify

Beta header names and tool version strings (Chapters 10 to 12, 17, 22), the exact parameter shapes for adaptive thinking and effort (Chapter 4) and native structured outputs (Chapter 5), current prices and minimum cache sizes (Chapters 7 and 15), the Agent SDK message field names (Chapter 23), and the current names of Claude Code looping commands (Chapter 25). When something fails, the docs page for that feature is the source of truth, and the fix is usually a one-line change.